```
Import cv2
import numpy as np
import matplotlib.pyplot as plt
import urllib.request
import time

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# ============================================================
# 1. INPUT IMAGE
# ============================================================

url = "https://raw.githubusercontent.com/opencv/opencv/master/samples/d
urllib.request.urlretrieve(url, “input.jpg")

ing = cv2.inread("input.jpg")

if img is None:
    raise Exception("Input image could not be loaded“)

print(“Input image loaded successfully")
print(“Image size:“, img.shape)

# ============================================================
# 2. PREPROCESSING


start = time.perf_counter()

gray = cv2.cvtColor(

    cv2.COLOR_BGR2GRAY


blurred = cv2.GaussianBlur(
    gray,
    (5, 5),
    0


preprocessing_time = (
    time.perf_counter()   start
)*  1000

# ============================================================
# 3. FEATURE DETECTION
# ============================================================

start = time.perf_counter()

sift   =  cv2.SIFT_create()

kp1, desl = sift.detectAndCompute(
    blurred,
    None
```

```
detection_time = (
    time.perf_counter()    start
) * 1000

print(“\nFeature Detection“)
print ( "----------------------------------------------")
print ( "Numbe n o-fi featur'es : ",   ten( kp1) )
print ( "Des c nlpton shape : ",   des 1. s hape)
print(
    "Detection time: ",
    round (detect1on_t1me ,  3) ,




# 4. CREATE TRANSFORMED IMAGE


h,  w = gray . shape

center = (
    w // 2,
    h // 2


rot at ion_mat rix  = cv2. getRotat1onMatnlx2D(
    center,
    2O,
    1. 0


transformed = cv2.warpAffine(
    blurred,
    rot at ion  matr ix,
    (w,  h)


# ============================================================
# 5. FEATURE MATCHING


start = time.perf_counter()

kp2, des2 = sift.detectAndCompute(
    t r ans forned ,
    None


bf  = cv2. BFMatcher()

matches = bf.knnMatch(
    des1,
    des2,
    k=2


eood  matehes  = 11
```

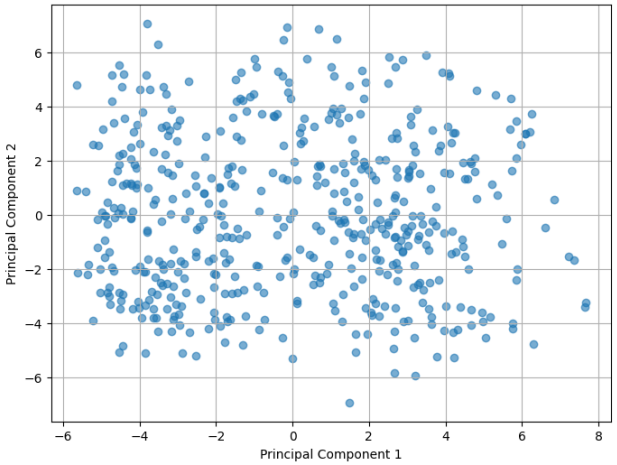
```
for m, n in matches:

    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

nat chi ng_time = (
    time.perf_counter()    start
) * 1000

print("\nFeature Matching")
print("----------------------------------------")
print("Features image 1:", len(kp1))
print("Features image 2:", len(kp2))
print("Total matches:", len(matches))
print("Good matches:", len(good_matches))
print(
    "Match i ng time:",
    round(match1ng_t1me, 3),



# ============================================================
# 6. FEATURE VISUALIZATION


feature_image = cv2.drawKeypoints(

    kp1,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS


plt.figure(figsize=(10, 7))

plt.imshow(
    cv2.cvtColor(
        feature_image,
        cv2.COLOR_BGR2RGB



plt.title(
    f"Detected SIFT Features = {len(kp1)}"


pit . axis("off")

plt.savefig("Q30_features.png")
plt.show()

# ============================================================
# 7. MATCH VISUALIZATION


match_image = cv2.drawMatches(
    blurred,
    kp1,
```

```
    kp2,
    good_matches[:50],
    None,
    flags=cv2.DrawMatchesFlags_N0T_DRAW_SINGLE_POINTS


plt.figure(figsize=(16, 8))

plt.imshow(
    cv2.cvtColor(
        match_image,
        cv2.COLOR_BGR2RGB



plt.title(
    f"Good Feature Matches = {len(good_matches)}"


plt.axis("off")

plt.savefig("Q30_matching.png")
plt.show()

# ============================================================
# 8. PCA DIMENSIONALITY REDUCTION


start = time.perf_counter()

# Limit number of descriptors
X = des1[:500]

# Standardize
scalen = StandandScalen()
X_scaled = scaler.fit_transform(X)

# PCA to 2 dimensions
pca = PCA(n_components=2)

X_pca = pca.fit_transform(
    X_scaled


pca_time = (
    time.perf_counter()  start
) * 1000

print("\nPCA")
print("-------------------------")
print("On1g1nal  dlmensions:",  X.shape[1 j)
print("Reduced dlmensions:", X_pca.shape[1])

variance = (
    sum(pca.explained_variance_ratio_)
    * 100
```

```
print(
    f“Variance retained: {variance:.2f}%“


print(
    f“PCA time: (pca_time:.3f} ms“


# ============================================================
# 9. PCA VISUALIZATION
# ============================================================

plt.figure(figsize=(8, 6))

plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    alpha=0.6


plt.xlabel(“Principal Component 1“)
plt.ylabel(“Principal Component 2“)

plt.title(
    "SIFT Features After PCA"


pit . grid(Tr‘ue )

plt.savefig(“Q30_PCA.png“)
plt.show()

# ============================================================
# 10. PERFORMANCE SUMMARY
# ============================================================

total_time = (
    preprocessing_time
    + detection_time
    + matching_time
    + pca_time


print ("\n=====================================")
print ("END-TO-END VISION SYSTEM SUMNARY" )
print("=====================================")

print(
    f"Preprocessing Time    {preprocessing_time:.3f} ms“


print(
    f“Feature Detection    {detection_time:.3f} ms“


print(
    f"Feature Matching     (matching_time:.3f} ms“
```

```
print(
    f“PCA Processing        {pca_time:.3-1} ms”


print(
    f“Total Processing      {total_time:.3f} ms“


print(
    f“Detected Features     {len(kp1)}“


print(
    f"Good Matches          {len(good_matches)}“


print(
    f“PCA Variance          {variance:.2f}%“


print("====================================")
```

```
Input image loaded successfully
Image size: (356, 493, 3)

Feature Detection
--------------------------
Number of features: 1192
Descriptor shape: (1192, 128)
Detection time: 279.514 ms

Feature Matching
--------------------------
Features image 1: 1192
Features image 2: 1141
Total matches: 1192
Good matches: 843
Matching time: 329.272 ms
```




```
==================================
END-TO-END VISION SYSTEM SUMMARY
==================================
Preprocessing Time : 9.115 ms
Feature Detection  : 279.514 ms
Feature Matching   : 329.272 ms
PCA Processing     : 141.776 ms
Total Processing   : 759.678 ms
Detected Features  : 1192
Good Matches       : 843
PCA Variance       : 15.02%
==================================
```